Chapitre 2 :
Git : Revenir en arrière (Rollback)

Plan



1. Parlons de HEAD
2. Revenir en arrière (1)
3. Correction des commits
4. Revenir en arrière (2) avec revert
5. TP 2

Parlons de HEAD

Rapport entre répertoire de travail et commits

- Git considère votre répertoire de travail comme étant en fait composé de :
  - ✓ l'ensemble des fichiers *suivis*, **dans leur version correspondant à UN SNAPSHOT COURANT** (un commit).
  - ✓ **PLUS** les modifications existant sur ces fichiers par rapport à cette version spécifique.
  - ✓ **PLUS** des fichiers non suivis (pour lesquels on n'a pas effectué de git add)

- **HEAD** : une référence (c'est à dire, un pointeur vers un commit) pour désigner le **'SNAPSHOT COURANT**'.
- **master :** une référence qui pointe vers la tête de l'historique.

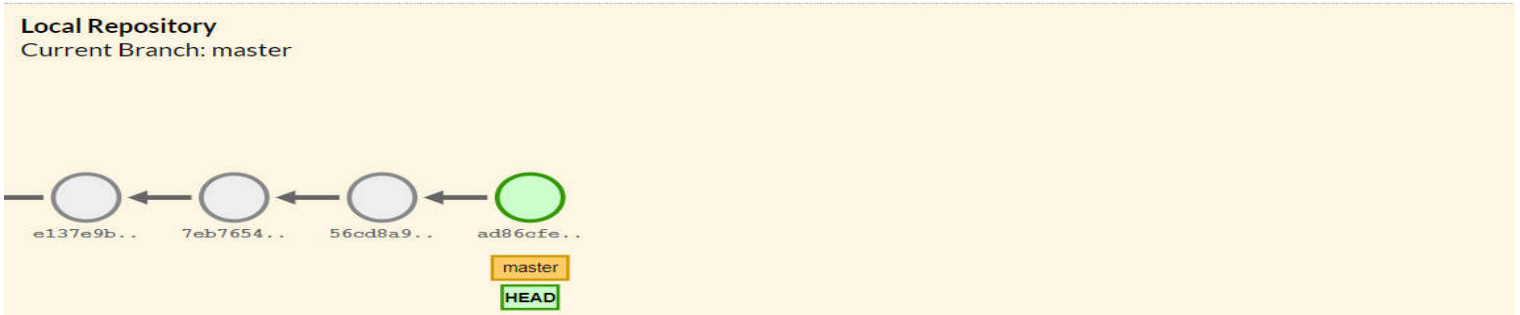
Parlons de HEAD

Manipulation de la référence HEAD

- La commande git commit effectue deux choses :

  1. Elle enregistre un *snapshot* ayant un lien de parenté avec le commit référencé par HEAD.

  2. Elle déplace la référence HEAD et la référence master sur ce nouveau commit.



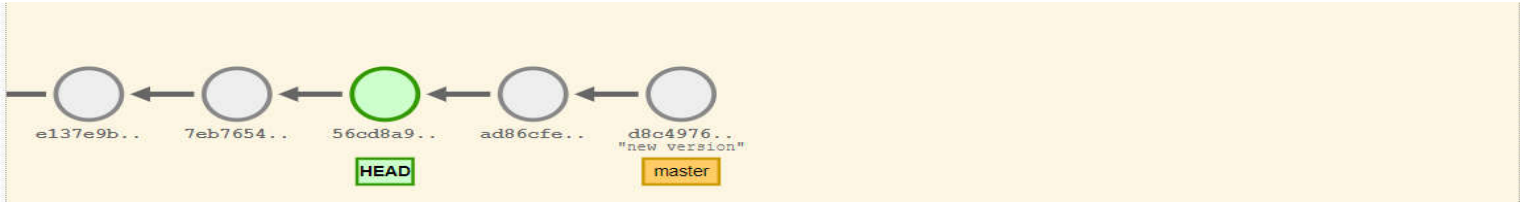
Suite à un commit portant le nom « new version » :



Parlons de HEAD

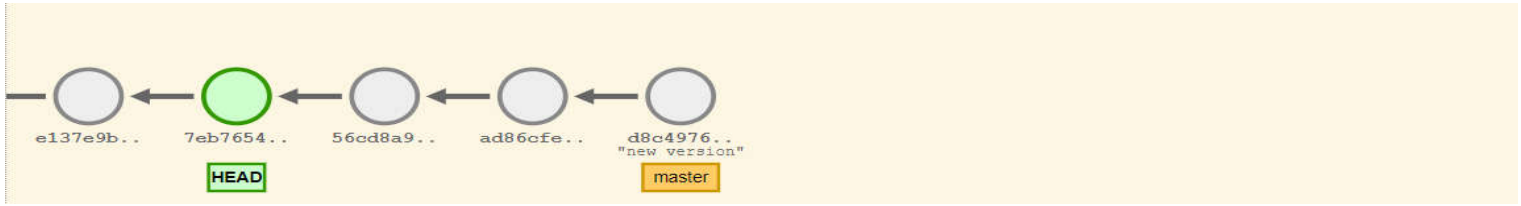
Manipulation de la référence HEAD

- **$git checkout 56cd8a9** : la référence HEAD se déplace sur le commit 56cd8a9

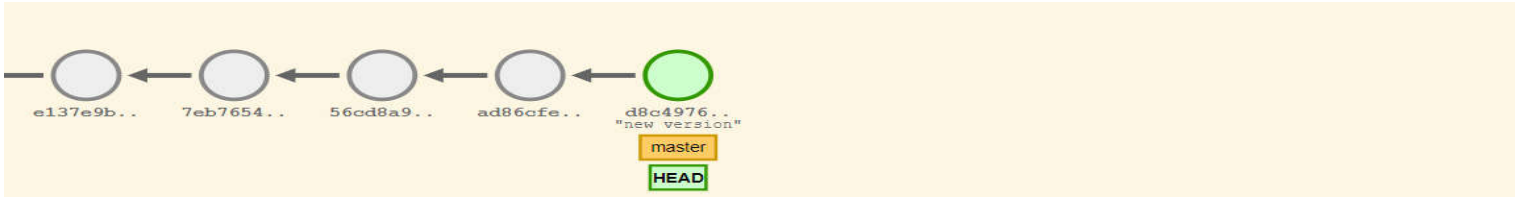


La commande **git checkout** va spécifiquement déplacer le HEAD vers un commit que vous lui précisez. En règle générale, elle ne peut être appliquée que si il n'y a pas de modifications en cours dans vos fichiers.

- **git checkout HEAD^** : la référence HEAD se déplace d'un cran vers le gauche (HEAD^ désigne le commit précédant HEAD)



- **git checkout master** : la référence HEAD se déplace vers la référence master, c'est-à-dire la tête de l'historique



Parlons de HEAD

Influence sur le répertoire de travail

- Rappel : votre répertoire de travail est constitué des fichiers suivis *dans leur version correspondant au commit référencé par 'HEAD'*
- Par conséquent, git checkout 56cd8a9, par exemple, va déplacer 'HEAD' sur le commit 56cd8a9, puis va faire en sorte que votre répertoire de travail soit constitué des fichiers *dans leur version correspondant au nouveau 'HEAD'*.
- Les fichiers sont donc modifiés, ajoutés, ou supprimés, pour que votre répertoire de travail soit dans l'état qui était le sien au moment du commit 56cd8a9...

Revenir en arrière (1)

- 1. Supposons qu'on vient de supprimer un fichier et de Commiter cette suppression :

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/TPs_GIT/TPRollback (master)
$ git commit -m "Suppression ProgPrincipal"
[master 888c1a4] Suppression ProgPrincipal
 1 file changed, 4 deletions(-)
 delete mode 100644 ProgPrincipal.py

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/TPs_GIT/TPRollback (master)
$ git log --oneline
888c1a4 (HEAD -> master) Suppression ProgPrincipal
2c9f609 1ere MAJ
03a51bd Ajout prog prncipal
427332e version 1
93393c9 version 0
```

- 2. On se rend compte qu'on a fait une mauvaise décision et on veut revenir à l'état précédant la suppression de ce fichier ➔ sol : utiliser la commande git checkout id_commit_précédent (2c9f609)

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/TPs_GIT/TPRollback (master)
$ git checkout 2c9f609
Note: switching to '2c9f609'.
```

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/TPs_GIT/TPRollback ((2c9f609...))
$ ls
ProgPrincipal.py  appFichierTicF.py  file.txt  notesModule.txt  sms.txt.txt
```

Revenir en arrière (1)

- Si vous affichez l'historique, vous pouvez constater que tous les commits suivants ont *disparu* (tout du moins en apparence). Pour les *retrouver*, **il faut replacer 'HEAD' sur la tête de l'historique enregistré dans le dépôt** :

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/TPs_GIT/TPRollback ((2c9f609...))
$ git log --oneline
2c9f609 (HEAD) 1ere MAJ
03a51bd Ajout prog prncipal
427332e version 1
93393c9 version 0
```

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/TPs_GIT/TPRollback ((2c9f609...))
$ git checkout master
Previous HEAD position was 2c9f609 1ere MAJ
Switched to branch 'master'

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/TPs_GIT/TPRollback (master)
$ git log --oneline
888c1a4 (HEAD -> master) Suppression ProgPrincipal
2c9f609 1ere MAJ
03a51bd Ajout prog prncipal
427332e version 1
93393c9 version 0
```

Revenir en arrière (1)

- À retenir : la commande git checkout id_commit permet de revenir en arrière mais sans pouvoir modifier.
- Pour récupérer le fichier ProgPrincipal.py, on doit taper :

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/TPs_GIT/TPRollback (master)
$ ls
appFichierTicF.py  file.txt  notesModule.txt  sms.txt.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/TPs_GIT/TPRollback (master)
$ git checkout 2c9f609 ProgPrincipal.py
Updated 1 path from a0027ec
```

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/TPs_GIT/TPRollback (master)
$ ls
ProgPrincipal.py  appFichierTicF.py  file.txt  notesModule.txt  sms.txt.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/TPs_GIT/TPRollback (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   ProgPrincipal.py
```

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/TPs_GIT/TPRollback (master)
$ git commit -m "restauration ProgPrincipal.py"
[master 8b63f89] restauration ProgPrincipal.py
 1 file changed, 4 insertions(+)
 create mode 100644 ProgPrincipal.py

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/TPs_GIT/TPRollback (master)
$ git log --oneline
8b63f89 (HEAD -> master) restauration ProgPrincipal.py
888c1a4 Suppression ProgPrincipal
2c9f609 1ere MAJ
03a51bd Ajout prog prncipal
427332e version 1
93393c9 version 0
```

Correction des commits

Annuler des commits

- Si on décide d'annuler notre dernier commit, il faut le supprimer de l'historique. Cette opération peut être réalisée avec la commande $ git **reset** HEAD^.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/TPs_GIT/TPRollback (master)
$ git log --oneline
8b63f89 (HEAD -> master) restauration ProgPrincipal.py
888c1a4 Suppression ProgPrincipal
2c9f609 1ere MAJ
03a51bd Ajout prog prncipal
427332e version 1
93393c9 version 0

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/TPs_GIT/TPRollback (master)
$ git reset HEAD^

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/TPs_GIT/TPRollback (master)
$ git log --oneline
888c1a4 (HEAD -> master) Suppression ProgPrincipal
2c9f609 1ere MAJ
03a51bd Ajout prog prncipal
427332e version 1
93393c9 version 0
```

Correction des commits

Annuler plusieurs commits

git reset permet aussi de **supprimer plusieurs commits**. Pour cela il y a plusieurs possibilités :

- Pour supprimer les 'n' derniers commits, il suffit de suffixer le caractère '^' n fois à la référence 'HEAD'. Par exemple pour supprimer les 5 derniers commits :

  $ git **reset** HEAD^^^^^

- Notation compressée :
  $ git **reset** HEAD~5

- Pour supprimer tous les commits après le commit ayant l'id 'f1fa39c' :
  $ git **reset** f1fa39c

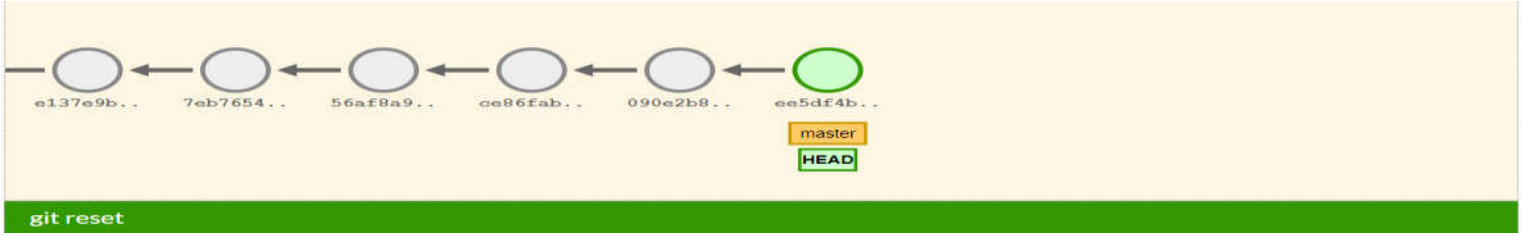
- Pour supprimer le commit ayant l'id 'f1fa39c' et ceux qui le suivent (et donc revenir *avant* le commit 'f1fa39c') :
  $ git **reset** f1fa39c^

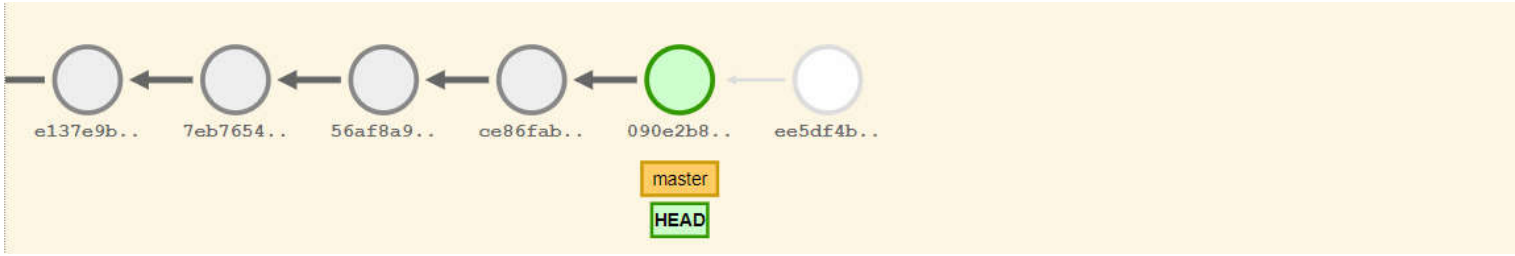
Correction des commits

Annuler des commits

- git checkout HEAD^ : déplacement de la référence **HEAD** mais sans *toucher* à l'historique enregistré (et **synchronisation** du répertoire de travail)

- git reset HEAD^ : déplacement de la référence de la tête de l'historique (**master**), ce qui force un déplacement de **HEAD** (mais **sans modification** du répertoire de travail





git reset va donc déplacer en arrière la référence qui *pointe* sur le dernier commit de l'historique (le **master**). C'est comme si ce commit n'avait jamais été effectué

Correction des commits

Annuler des commits

- **git checkout** agit **sur le répertoire de travail** : le répertoire de travail est restauré dans l'état enregistré dans le commit sélectionné.

  - ✓ Pour cela, Git peut modifier vos fichiers, et en ajouter ou supprimer d'autres (Git ne touche cependant pas aux fichiers non suivis).

  - ✓ Cette commande ne modifie pas l'historique enregistré dans le dépôt (la référence **master** n'est pas modifiée). On peut donc revenir à la dernière version avec git checkout master.

- **git reset** agit **sur l'historique** : la référence de tête (**master**) est déplacée, ce qui supprime des commits de l'historique.

Correction des commits

Annuler des commits

- **git reset** agit **sur l'historique** : la référence de tête (**master**) est déplacée, ce qui supprime des commits de l'historique.

  •**Avec l'option --soft :** le contenu du répertoire de travail n'est pas modifié : les différences entre le répertoire de travail et le nouveau commit de tête sont considérées comme des modifications placées dans l'index, mais non encore validées.
  Résultat : on revient juste au moment où l'on avait fait le git add mais sans avoir encore fait le commit.

  •**Dans sa version de base (option --mixed par défaut) :** le contenu du répertoire de travail n'est toujours pas modifié : les différences entre le répertoire de travail et le nouveau commit de tête sont considérées comme non indexées.
  Résultat : on revient un cran plus tôt, juste au moment où l'on avait modifié les fichiers mais sans avoir encore fait le git add.

  •**Dans sa version --hard :** le répertoire de travail est également synchronisé avec le nouveau commit de tête.
  Résultat : on revient un cran de plus en arrière, avant le moment où l'on avait modifié les fichiers.

Correction des commits

Récapitulons

- **git checkout <commit_id>** est une commande utilisée très régulièrement lorsque l'on travaille avec des *branches*, comme vous le verrez bientôt. C'est l'usage principal de cette commande.

- **git checkout <un_fichier>** est occasionnellement utilisée pour supprimer des modifications que l'on vient d'ajouter à un fichier.

- **git reset** est rarement utilisée. Elle ne sert, à priori, que lorsque l'on décide d'abandonner un travail en cours et de revenir en arrière.

ATTENTION : Comme pour toute commande modifiant l'historique, git reset ne doit pas être utilisé sur des historiques partagés avec d'autres contributeurs.

Revenir en arrière avec revert

- **git revert** : cette commande est utilisée pour annuler les modifications partagées en créant de nouveaux commits. Les nouveaux commits inversent les modifications précédentes, de sorte que l'historique de commits reste cohérent. Cela signifie que la commande **git revert** est une opération sûre et ne supprime pas les modifications précédentes.

- En résumé, si vous souhaitez annuler des modifications non partagées, utilisez **git reset**. Si vous souhaitez annuler des modifications partagées sans supprimer l'historique des commits, utilisez **git revert**.

Revenir en arrière avec revert

Exemple

- Supposons que vous ayez un projet Git avec trois commits, numérotés 1, 2 et 3, comme suit:
  Commit 1 -> Commit 2 -> Commit 3

- Supposons également que vous avez modifié le contenu du projet et avez effectué un nouveau commit:
  Commit 1 -> Commit 2 -> Commit 3 -> **Commit 4**

- Maintenant, supposons que vous souhaitiez annuler le dernier commit (**Commit 4**) en raison d'une erreur. Si vous utilisez **git reset**, vous pouvez supprimer le commit 4 et revenir à l'état précédent du projet en utilisant la commande suivante: git reset --hard HEAD~1

➔ Cela supprimera le dernier commit et réinitialisera l'état du projet au commit 3. Cependant, cette commande supprime définitivement le commit 4 et toutes les modifications qui y sont associées. Si vous avez partagé ce commit avec d'autres personnes, cela peut causer des problèmes.

Revenir en arrière avec revert

Exemple

- Si vous utilisez **git revert**, vous pouvez annuler le commit 4 en créant un nouveau commit qui annule les modifications apportées dans le commit 4 en utilisant la commande suivante: git revert HEAD

- Cela créera un nouveau commit (numéroté 5 dans cet exemple) qui annule les modifications apportées dans le commit 4, tout en préservant l'historique des commits. **Cela signifie que si d'autres personnes ont déjà récupéré le commit 4, elles pourront toujours le récupérer, mais verront que le commit 5 l'annule.**

- En résumé, utilisez **git reset** si vous souhaitez supprimer définitivement des commits et **git revert** si vous souhaitez annuler des modifications tout en préservant l'historique des commits.

TP2 : Rollback